

Team Application Exercise 1 (tAPP)

Instructions: Work on problem 1 on your own to apply the concepts you learned in the pre-reading material to solve problems. Then discuss your code with your team members and present one solution as a team. Follow the same process for all the problems. Swap your team's solution with another team for feedback.

Student Tasks

Σ Task 1: String slicing: favourite song

Write a program that asks the user to enter the first line of a favourite song. Display the number of characters in the line. Then ask the user for a starting index and an ending index, and display only that section of the text using slicing.

Σ Task 2: Umbrella or no umbrella

Write a program that asks the user whether it is raining. Convert the answer to lowercase so that different capitalisation does not affect the decision. If the answer is yes, ask whether it is windy. If it is both raining and windy, display "It's too windy for an umbrella". If it is raining but not windy, display "Take an umbrella". If it is not raining, display "Enjoy your day".

A Task 3: Eligibility checker

A short training course is open only to staff who are currently active and have a score of at least 70. Write a program that asks for the staff member's status and score, then displays either "Eligible" or "Not eligible". Treat ACTIVE, Active and active as equivalent.

A Task 4: Debug and improve the program

The following program is intended to classify a mark as Distinction (70 or above), Pass (50 to 69), or Fail (below 50). It contains errors. Identify the problems, write a corrected version, and explain each correction.

Starter code

```
mark = input("Enter mark: ")

if mark >= 70
    print("Distinction")
elif mark >= 50:
    print("Pass")
else
    print("Fail")
```

N Task 5: Build and use a simple Employee class

Create an Employee class. Each Employee object should store a name and a score as instance attributes. Add an instance method called report() that prints the employee's name and score. Create two different Employee objects and call report() for both.

Starter code

```
class Employee:
    # complete the class
    pass
```

N Task 6: Combine OOP and decision making

Extend your Employee class so that it also stores whether the employee is active. Add a method called eligible_for_training() that returns True only when the employee is active and has a score of at least 70. Then create at least three Employee objects and use an if statement to print an appropriate eligibility message for each employee.

S Task 7: Inheritance challenge

Create a Manager class that inherits from Employee. A Manager should inherit the existing employee behaviour and add a method called lead_team() that prints a short message. Create a Manager object, call an inherited method, call lead_team(), and check whether the object is also an instance of Employee using isinstance().

S Task 8: Team integration challenge

Design a small program that combines the ideas from this week's material. The program should create at least three Employee or Manager objects, store a name, score and active status for each, and report whether each person is eligible for training. At least one object must be a Manager. Your team should decide what code belongs inside the class and what code belongs outside it.

Your team solution should:

- use classes and objects appropriately;
- include at least one method that uses decision-making logic;
- avoid unnecessary duplicated code;
- produce readable output for all objects;
- be ready to explain one design decision your team made.